

Gym Management System

By 404 Gang

Team Members

Name :

ID :

Lingkan Sarkar

2252081177

Sabit Bin Bashar

2252081168

Israt Jahan

2252081173

Koushik Sarker Arup

2252081178

Riti Amin Simila

2252081224

Introduction and Objective

Introduction

The Gym Management System is a console-based software application designed to automate and streamline daily fitness center operations. The system manages member registrations, trainer allocations, customized workout routines, subscription packages, and billing records in a unified platform.

Objective

Member & Staff Management: Register, update, and manage records for gym members and fitness trainers.

Trainer Allocation: Assign personal fitness trainers to members according to training needs and workload.

Workout Routine Planning: Create, update, and assign personalized workout plans with detailed exercise routines.

Membership & Billing: Handle multi-tier membership packages, calculate total fees, track subscription expiry dates, and issue payment receipts.

Operational Monitoring: Track daily member attendance, monitor member activity history, and generate trainer workload reports.

OOP Concepts Used in this Project

Abstract Classes: GymEntity serves as a blueprint with abstract methods (GetSummary()) to prevent direct instantiation and enforce structure across child classes.

Inheritance: Person inherits from GymEntity, while both Member and Trainer inherit shared attributes and behaviors from Person.

Interfaces: IReportable establishes a standard contract requiring Member, Trainer, and Gym to implement the GetReport() method.

Polymorphism & Method Overriding: Person provides a virtual method GetRole() (or GetRoleName()), which Member and Trainer override to return their specific identity.

Function Overloading: Multiple versions of methods like RecordAttendance() and AddExercise() handle varying parameter sets (e.g., logging today's date vs. a custom date).

Constructors & Constructor Overloading: Classes like Member, Trainer, and Membership feature default and parameterized constructors to support flexible object initialization.

Copy Constructors: Implemented in Member, Membership, and WorkoutPlan to perform safe, independent deep copies of existing objects.

Operator Overloading: Overloaded == and != operators in Member enable custom equality checks based on unique Id rather than memory reference.

GymEntity (Abstract Class)

Purpose: Serves as the base abstract class for all identifiable entities in the gym system, ensuring a standard blueprint while preventing direct instantiation.

Properties:

- Id – Stores the unique identifier for the entity.
- Name – Stores the name of the entity.

Methods:

- GetSummary() – An **abstract method** that forces all derived classes to implement their own custom logic to display entity details.

Person Class

Purpose: Acts as a base class for human entities in the system (Member and Trainer), providing shared personal attributes and behaviors.

Inheritance: Inherits common identity properties (Id, Name) from GymEntity.

Properties:

- Age (stores the person's age).

Methods & Features:

- **Constructor:** Initializes Id, Name, and Age by passing identity fields to the base() constructor.
- **GetRole() / GetRoleName():** A virtual method that returns a default label like "Person", designed to be overridden by derived classes.
- **GetSummary():** Implements or overrides the summary method to display standard demographic details.

Member Class

Purpose: Represents gym members and manages their profiles, subscriptions, workout plans, and attendance.

Properties:

- CurrentMembership (stores active subscription package)
- AssignedTrainer (stores assigned personal trainer)
- Plan / CurrentWorkoutPlan (stores assigned workout plan)
- ExpiryDate (tracks subscription validity)
- AttendanceHistory (records check-in logs)

Methods & Features:

- **Constructors:** Default, parameterized, and copy constructor for object cloning.
- **RecordAttendance():** Function overloading to log today's check-in or a custom date.
- **AssignMembership():** Assigns a package and calculates expiry date.
- **GetRole() & GetReport():** Method overriding and interface implementation.
- **Operator Overloading:** Overloads == and != to compare members by unique Id.

Trainer Class

Purpose: Represents fitness trainers and manages their assigned gym members.

Inheritance & Interfaces: Inherits from the Person base class and implements the IReportable interface.

Properties:

- AssignedMembers (stores the list of members assigned to the trainer).

Methods & Features:

- **Constructors:** Constructor overloading to initialize trainers with full demographic info or a default age.
- **GetWorkload():** Virtual method that returns the total count of assigned members.
- **GetRole():** Method overriding that returns "Fitness Trainer".
- **GetReport():** Interface implementation that generates a summary of the trainer's workload.

Membership Class

Purpose: Represents subscription packages, handles pricing and tax calculations, and generates billing receipts.

Properties & Fields:

- Name (stores package tier name, e.g., Basic, Standard, Premium).
- MonthlyFee (stores monthly subscription cost).
- DurationMonths (stores validity period in months).
- TaxRate / DefaultDiscountRate (static field for system-wide tax or discount rates).

Methods & Features:

- **Constructor Overloading:** Provides default package initialization as well as fully parameterized constructors.
- **Copy Constructor:** Duplicates an existing membership configuration safely.
- **CalculateTotalFee() / CalculateFee():** Overloaded methods to compute total cost with duration, taxes, or custom discount vouchers.
- **GenerateReceipt():** A static utility method that formats and produces a standard payment receipt without needing a dedicated billing instance.

Work Process

Initialization & Setup: The gym instance is initialized with default membership tiers (Basic, Standard, Premium), registered fitness trainers, and pre-configured workout routines.

Member Registration: New members are registered into the system with their unique ID, name, and age.

Service Allocation: Each member is assigned a specific membership package, a dedicated personal trainer, and a personalized workout plan.

Daily Operations:

- Members record their daily check-in attendance.
- The system computes membership fees, applies applicable discounts, and generates standard payment receipts.
- Workout plans are updated with custom exercises, sets, and repetitions as needed.

Monitoring & Reporting: The system continuously evaluates membership expiry dates, tracks individual member check-in history, and generates trainer workload reports.

Output

===== PAYMENT RECEIPT =====

Member Name : Koushik

Package : Premium 6-Month

Amount Paid : tk 400.00

Date Issued : 2026-08-22

=====

--- MEMBERSHIP STATUS TRACKER ---

Koushik (ID: M1) -> ACTIVE (Expires: 2027-02-22)

Israt (ID: M2) -> ACTIVE (Expires: 2026-09-22)

Simila (ID: M3) -> ACTIVE (Expires: 2026-09-22)

Dinal Trump (ID: M4) -> ACTIVE (Expires: 2026-09-22)

--- TRAINER WORKLOAD REPORT ---

Trainer [Sabit] | Workload: 1 member(s) assigned.

--- MEMBER ACTIVITY REPORT ---

Member [Koushik, ID: M1] | Status: Active until 2027-02-22 | Visits: 2 | Plan: Fat Shredder

Member [Israt, ID: M2] | Status: Active until 2026-09-22 | Visits: 1 | Plan: Mass Builder

Member [Simila, ID: M3] | Status: Active until 2026-09-22 | Visits: 0 | Plan: None

Member [Dinal Trump, ID: M4] | Status: Active until 2026-09-22 | Visits: 0 | Plan: None

Operator Overload Equality Test: True

Project Result

Automated Member Management: Successfully registered members, assigned them to trainers, and tracked individual workout routines without data duplication.

Accurate Billing & Invoicing: Seamlessly calculated total subscription costs—including tax and discount vouchers—and generated standardized billing receipts using static methods.

Real-Time Expiry & Attendance Tracking: Correctly monitored membership expiration statuses and logged member check-in history.

Trainer Workload Visibility: Provided immediate reporting on active client loads per trainer to ensure balanced coaching assignments.

Robust OOP Implementation: Verified full functionality of core OOP principles, including deep object copying via copy constructors and unique ID comparisons via overloaded equality (==) operators.

Conclusion

Successful Implementation: Successfully built a modular, fully functioning console-based Gym Management System in C#.

OOP Integration: Effectively implemented and demonstrated major Object-Oriented Programming concepts, including abstraction, inheritance, interfaces, polymorphism, overloading, copy constructors, and operator overloading.

Practical Problem Solving: Automated essential fitness center tasks, including member registrations, trainer allocations, workout management, attendance logging, and billing calculations.

Maintainability & Scalability: Designed a clean, decoupled architecture that makes adding new facilities, membership tiers, or reporting features simple and efficient.

•

Thank You